

A Minimal VPN Tunnel

Design, implementation and experimental evaluation

Team T005 · IIT Gandhinagar

Aayush Kuloor	24110004	Anvit Aggarwal	24110048
Dhru Harsh Darshakbhai	24110107	Raj Amit Modi	24110282

Abstract

This project implements **svpn**, a point-to-point IPv4 tunnel between two Raspberry Pi 5 hosts. A C userspace process reads packets from Linux TUN interfaces and transports them over UDP. Encrypted mode adds signed ephemeral key exchange, pinned peer identities, ChaCha20-Poly1305 and a 64-packet replay window. In five interleaved TCP trials per direction, encrypted sender throughput is 48.73 ± 3.61 Mbps from A to B and 45.01 ± 1.89 Mbps in reverse. Encryption adds approximately five percentage points of process CPU. Packet experiments distinguish small-datagram limits and loaded latency from cryptographic cost.

1 Objectives and architecture

The objective is to forward ICMP, TCP and UDP with explicit routing and packet protection. Direct Wi-Fi provides the baseline; plaintext tunnelling isolates forwarding cost; encrypted tunnelling adds protection. Two Raspberry Pi 5 boards running Debian 13 carry the measured traffic; a Fedora laptop orchestrates experiments.

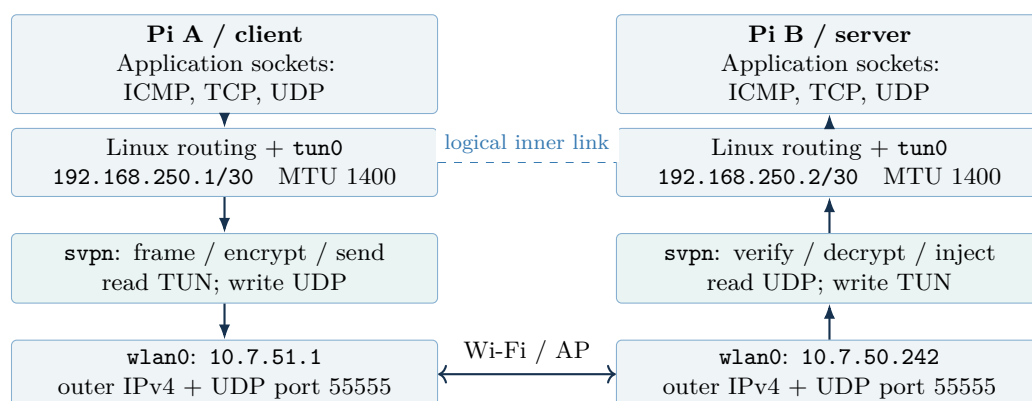


Figure 1: Logical and physical paths. Solid vertical arrows show one A-to-B packet; the reverse direction uses the same components.

1.1 Packet path and transport choice

A packet addressed to 192.168.250.2 enters A's `tun0` through Linux routing. Reading TUN returns the inner IP packet; **svpn** frames and optionally encrypts it, then sends a UDP datagram to B's Wi-Fi address. B validates and decrypts the datagram and writes the recovered packet to TUN, where Linux delivers it to the local socket. TUN exposes IP packets rather than Ethernet frames, and `IFF_NO_PI` omits the extra packet-information prefix [1].

UDP supplies datagrams without guaranteed delivery or duplicate protection [2]. Each inner packet becomes one outer datagram; DATA is not retransmitted by the tunnel. This leaves loss recovery to inner TCP and avoids adding an outer reliable byte stream. Setup scripts create and address the persistent TUN device; the daemon does not edit routes or firewall rules. The inner /30 route leaves the default route intact.

1.2 Scope and security boundary

The threat model allows an adversary to capture, modify, inject or replay outer packets, but assumes neither endpoint is compromised. Encrypted mode protects inner contents; outer addresses, port, lengths, timing and direction remain visible. The design supports one peer and host-to-host IPv4, with no NAT, gateway, roaming or automatic rekeying. Availability is outside its scope.

2 Wire protocol and session establishment

SVPN serializes a 28-byte header explicitly in big-endian order, avoiding native-structure padding. Its fields are magic (4 bytes), version (1), type (1), flags (2), session ID (8), sequence (8), plaintext length (2) and reserved (2). Version is 1; flags and reserved must be zero. Types 1/2 are CLIENT_HELLO/SERVER_HELLO, 3 is DATA and 4 is CLOSE, which is parsed but has no active close exchange.

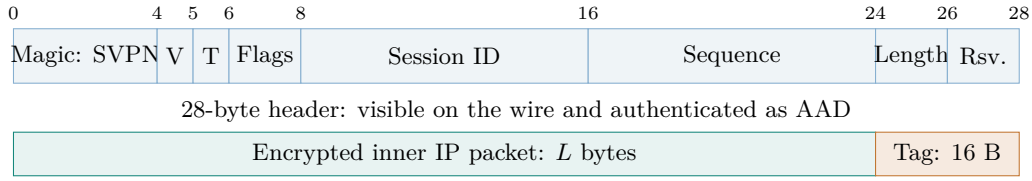


Figure 2: Encrypted DATA layout. Header positions are byte offsets; the body is schematic and not drawn to byte scale. V: version; T: type; Rsv.: reserved.

DATA requires $1 \leq L \leq 1400$ inner bytes and an exact body length: L in plaintext mode or $L + 16$ when encrypted. Truncation, trailing bytes and unsupported fields are rejected. ChaCha20-Poly1305-IETF uses a 12-byte nonce and a 16-byte tag [3]; the SVPN header is authenticated additional data (AAD), while the whole inner IP packet is encrypted. Assuming a 20-byte outer IPv4 header, the outer packet length is

$$B_{\text{plain}} = L + 28 + 8 + 20 = L + 56, \quad B_{\text{encrypted}} = L + 72.$$

At TUN MTU 1400, encrypted outer packets are 1472 bytes, below a 1500-byte outer MTU. The 72-byte encapsulation is 5.14% of a full inner packet; encryption alone adds 16 bytes. A UDP payload of 1372 bytes fits exactly after its 8-byte UDP and 20-byte IPv4 headers. These byte ratios exclude link-layer work and are not throughput predictions.

2.1 Pinned identities and signed ephemeral keys

The handshake is a custom two-message protocol built from standard cryptographic primitives, not a standardized handshake. Each host pins its peer's Ed25519 public key and signs using its local identity secret [4]. Fresh ephemeral keys feed libsodium's complementary client/server `crypto_kx` routines [5]:

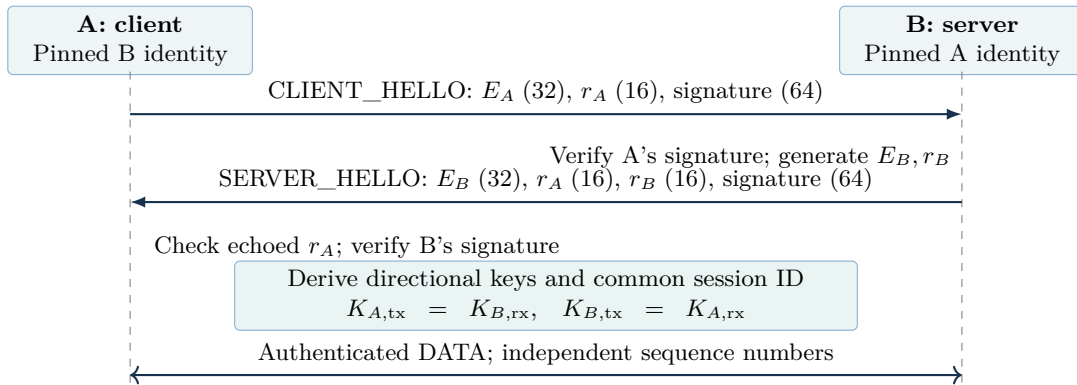


Figure 3: Two-message setup. Field lengths are bytes and exclude the 28-byte common header. E_A, E_B are ephemeral public keys; r_A, r_B are random values.

The client signs SVPN-CHLO-v1, version, r_A and E_A ; the server signs SVPN-SHLO-v1, version, r_A, r_B, E_A and E_B , in order. This binds the response to the initiating random and both ephemeral keys. The client retries every 500 ms within 10 s. A matching repeated hello elicits the retained server response; invalid signatures are ignored while waiting for valid setup.

2.2 Session identifier and nonce

Let h be BLAKE2b with a 32-byte output [6]. The session ID is the first eight bytes of $h(r_A \| r_B \| E_A \| E_B)$. For direction label $d = A2B$ or $B2A$,

$$N = \text{prefix}_4(h(d \| \text{sid})) \parallel \text{BE64}(\text{sequence}).$$

Directional keys are distinct; sequences start at one and never wrap. Restarts generate fresh keys. DATA replay protection does not prevent replayed signed hellos from consuming responder setup state.

3 Implementation and validation

The daemon is a single-threaded C17 program with one TUN descriptor and one UDP socket. After setup, `poll()` waits for either input. Fixed buffers bound packet storage. Protocol, cryptographic, handshake, replay and device-I/O routines are separated; the main loop coordinates forwarding and counters. Private key loading checks file type, size, ownership and permissions. Identity secrets are cleared after setup; session keys are cleared on shutdown.

3.1 Receive checks and replay window

The encrypted DATA path checks the source IP/port, header, exact body length and session ID, then authenticates and decrypts. Only successful authentication can advance replay state. Accepted packets enter TUN; failures increment categorized drop counters. SIGINT/SIGTERM request shutdown and print statistics.

Each receive direction maintains the highest sequence H and a 64-bit bitmap. Bit k records $H - k$. For $s > H$, the bitmap shifts left by $s - H$ and sets the low bit; a jump of at least 64 clears old history first. For $s \leq H$, a packet is eligible only if $H - s < 64$ and its bit is clear. Sequence zero and duplicates are rejected. Plaintext mode does not enforce this window.

Example: highest accepted sequence $H = 100$

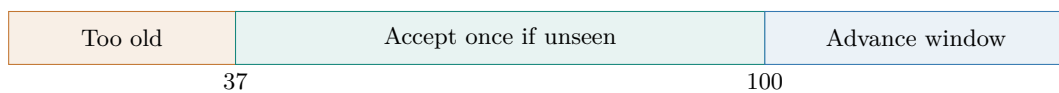


Figure 4: The valid reordering interval is $[H - 63, H]$, inclusive. With $H = 100$, sequence 36 is too old; sequence 37 is still eligible if unseen.

3.2 Functional and negative tests

Test group	Main checks
Protocol	Exact encoding, endian conversion, truncated headers, invalid fields, body mismatch, trailing bytes and oversize DATA.
Cryptography	Matching directional keys; AEAD round trips; altered ciphertext, tag, AAD, key and nonce must fail.
Handshake	Valid signatures, modified ephemeral keys/randoms, wrong pinned identity, client-random echo and session agreement.
Replay	First packet, monotonic sequence, duplicate, bounded reorder, old packet, large jump, zero and integer edges.
Integration	ICMP/TCP/UDP through TUN in both modes; wrong peer identity must prevent setup and inner ping.

Recorded deployments report 347 passing assertions initially and 349 later, with zero failures. Positive/negative smoke tests and sanitizer runs also passed on both Pis. AddressSanitizer and UndefinedBehaviorSanitizer provide complementary memory and undefined-behaviour checks [7, 8]. These results describe the existing deployments, not a fresh testbed run.

3.3 Confidentiality at the interface boundary

The known marker `SVPN_DEMO_SECRET_6F91C2A7` was searched for in the captured bytes of each packet. All three retained captures contain ten packet records:

Encrypted / <code>tun0</code> 1 of 10 packets contains the marker	Encrypted / <code>wlan0</code> 0 of 10 packets contains the marker	Plaintext / <code>wlan0</code> 1 of 10 packets contains the marker
--	---	---

Figure 5: Packet-capture marker search at the encryption boundary.

The marker is visible before encryption at TUN and in the plaintext outer control, but absent on the encrypted outer path. This demonstrates the expected boundary without proving general security or metadata privacy. Invalid DATA is dropped, although sustained floods can still exhaust receive, crypto or logging resources.

4 TCP throughput and CPU cost

The interleaved TCP experiment uses five 20 s TCP trials per mode and direction, with a 3 s omit period. Each trial runs direct, plaintext and encrypted modes in that order, reducing separation in time between comparisons. Throughput is measured with iperf3 [9]. TCP results use sender Mbps; receiver means are slightly different. Error bars show sample standard deviation across five trials, not confidence intervals.

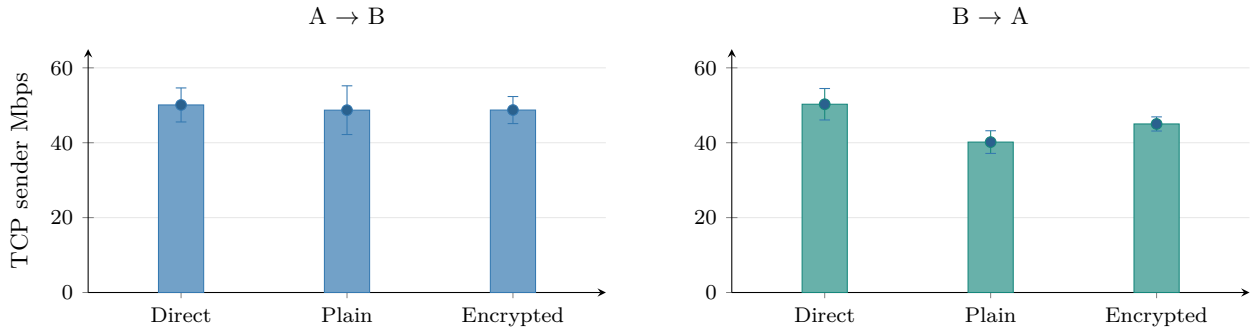


Figure 6: Interleaved TCP benchmark: mean $\pm$ sample SD, $n = 5$ for every bar.

Direction	Direct (Mbps)	Plaintext (Mbps)	Encrypted (Mbps)
A $\rightarrow$ B	50.09 $\pm$ 4.54	48.70 $\pm$ 6.51	48.73 $\pm$ 3.61
B $\rightarrow$ A	50.28 $\pm$ 4.20	40.17 $\pm$ 3.03	45.01 $\pm$ 1.89

For mean rates D, P, E , the tunnel, incremental encryption and total VPN changes are $100(D-P)/D$, $100(P-E)/P$ and $100(D-E)/D$. In A $\rightarrow$ B these are 2.78%, -0.06% and 2.72%; in B $\rightarrow$ A they are 20.10%, -12.06% and 10.47%. A negative incremental value means encrypted was faster in this sample. It does not establish that encryption improves the path, and direct-to-encrypted includes much more than cipher work.

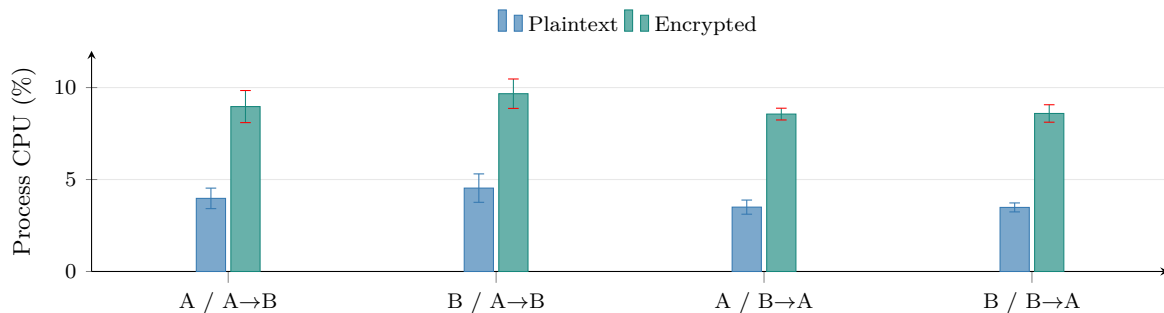


Figure 7: Per-process CPU on each Pi, grouped by traffic direction ($n = 5$). The encrypted-minus-plaintext differences are 4.99, 5.13, 5.06 and 5.11 percentage points using unrounded means.

The CPU difference is more consistent than the throughput difference. It includes nonce hashing, AEAD and receive replay work, not an isolated cipher microbenchmark. Five trials and a fixed mode order cannot eliminate Wi-Fi variation. Recorded throttle flags had historical sticky bits set, while snapshot live bits were zero; snapshots cannot exclude every transient event during a run.

5 UDP packet size and offered load

The packet experiment of 30 August complements the TCP results with 10 s UDP trials, two repeats, and interleaved modes. Here goodput means receiver application Mbps from iperf3, not the configured offer. The 64-byte payload is an intentionally synthetic stress test: at a 20 Mbps application-payload offer it requests approximately 39,063 packets/s. The 1372-byte payload is the bulk-transfer endpoint because it is the largest UDP payload that fits the 1400-byte inner MTU; the same offer requires only 1,822 packets/s.

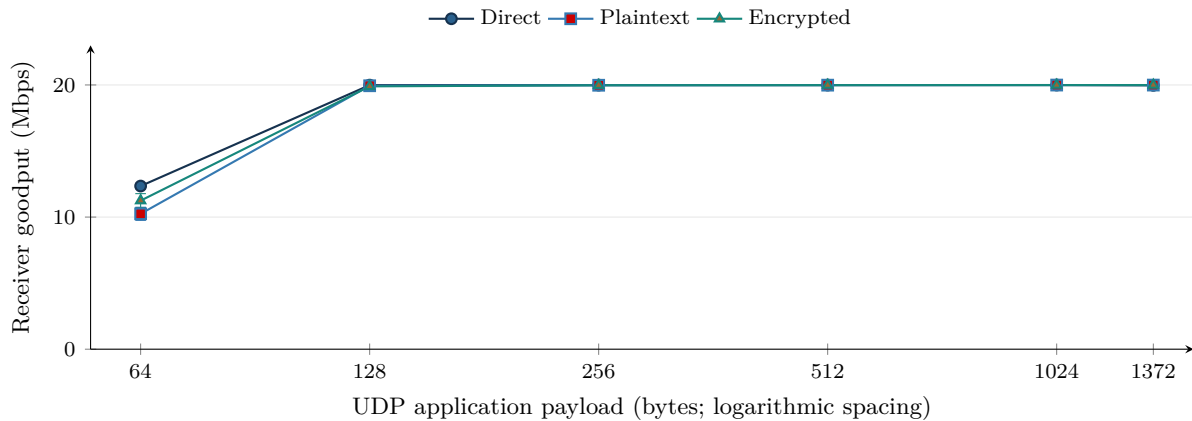


Figure 8: A→B goodput at a configured 20 Mbps offer, mean $\pm$ sample SD ($n = 2$). The 64-byte point is a synthetic packet-rate stress test; 1372 bytes is the largest UDP payload permitted by the inner MTU. Most error bars are smaller than the markers.

5.1 The 64-byte synthetic stress test

The 64-byte result is retained specifically to expose packet-rate behaviour; it is not presented as a representative VPN workload. Direct mode actually sends only 12.37 Mbps and receives 12.35 Mbps, with effectively zero reported loss. Plaintext and encrypted modes send about 20 Mbps but receive 10.25 and 11.24 Mbps, with 47.75% and 43.50% reported loss. The low direct loss therefore does not mean that direct mode delivered the requested 20 Mbps: its achieved send rate is already lower. These measurements reveal a small-packet ceiling somewhere in the traffic generator, kernel, forwarding path or wireless network, but they do not identify a single bottleneck or show that encryption caused the ceiling.

From 128 through 1372 bytes, A→B delivers roughly 20 Mbps in all modes, with at most 0.43% mean loss. The reverse direction has the same broad shape, but direct 128-byte traffic is an exception: 18.25 ± 2.46 Mbps and $7.99 \pm 11.30\%$ loss. That noisy pair of trials should not be omitted when generalizing the result. Encrypted CPU on A/B decreases from 13.40/15.15% at 64 bytes to 3.50/3.10% at 1372 bytes, consistent with the reduced packet rate.

5.2 Increasing the offer beyond delivered capacity

The A→B rate sweep compares the stress-test endpoint with the bulk-transfer endpoint. With 64-byte payloads, encrypted goodput is 5.00 and 9.95 Mbps at 5 and 10 Mbps offers. Higher offers leave delivery near 11–12 Mbps while reported loss rises from 40.43% at 20 Mbps to 85.26% at 80 Mbps. This is the expected consequence of offering more tiny datagrams than the path can deliver; it does not make 64 bytes a useful operating point for bulk traffic.

With 1372-byte payloads, encrypted delivery tracks the offer through 40 Mbps and reaches 45.26 Mbps at an 80 Mbps offer, with 40.88% mean loss. The intermediate-size sweep is equally important: from 128 bytes onward, all three modes sustain approximately 20 Mbps at the fixed offer. In a separate 100 Mbps max-effort test, encrypted receiver goodput is 11.44 Mbps at 64 bytes and 37.70 Mbps at 1372 bytes. Those runs occurred at different times and are not combined into one capacity curve. Because the tunnel has no outer congestion control, excess UDP offers primarily increase loss.

6 Latency, queueing and MTU behaviour

The ping size sweep uses 30 probes per mode and payload size at 0.2 s intervals. Encrypted mean RTT ranges from 9.07 to 10.75 ms across the reported nonzero sizes. The observations do not show a consistent monotonic encryption penalty. This is a short network experiment, so it cannot isolate sub-millisecond cipher execution time from scheduling and wireless delay.

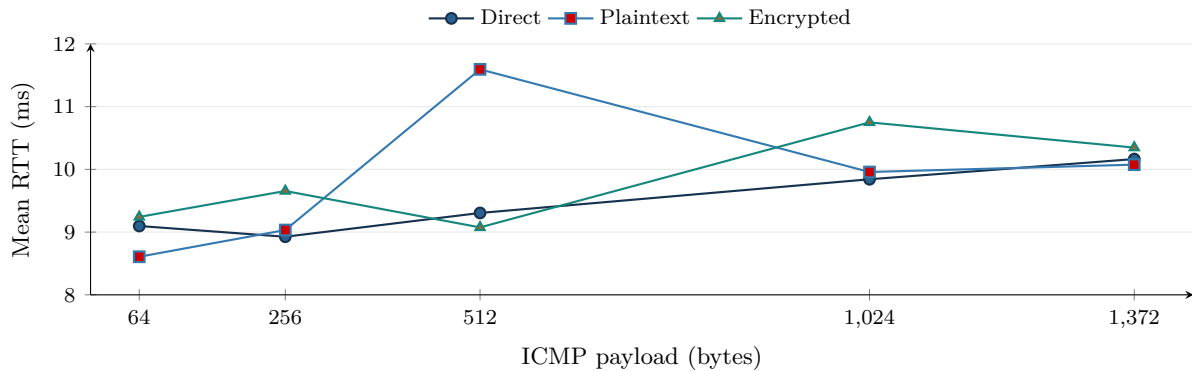


Figure 9: Idle RTT size sweep, A→B. Each point is a 30-probe session average; no between-session error bars are available. The vertical axis starts at 8 ms to show the variation.

The zero-byte ICMP payload probes returned replies, but this iputils build omitted timing output; RTT is therefore unavailable, not zero. Direct mode lost one of 30 probes at the 512-byte setting (3.33%), while the tunnel modes reported no loss in that sweep. A 1400-byte ICMP payload creates a 1428-byte inner IP packet after the ICMP and IPv4 headers are added. With `ping -M do -s 1400`, both tunnel modes correctly return “Message too long” at MTU 1400. Direct mode succeeds because 1428 bytes fits its 1500-byte interface MTU.

6.1 Loaded latency is much less stable than idle latency

Two further repetitions compare idle pings with pings sent during a 15 s TCP load. Mean idle RTT is 8.96, 8.68 and 8.70 ms for direct, plaintext and encrypted modes; corresponding loaded averages are 163.69, 150.01 and 241.37 ms.

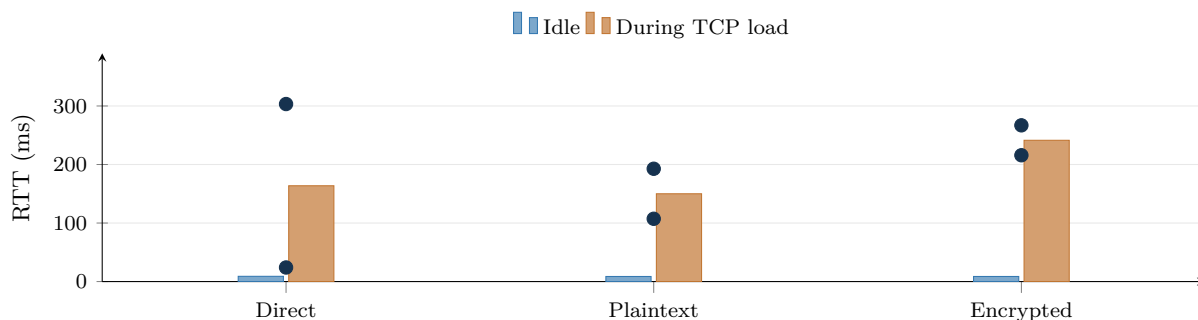


Figure 10: Mean RTT for the two load-test repetitions. Dark dots show the individual loaded-session averages, making the small sample and variability visible.

The two direct loaded averages are 303.32 and 24.06 ms, a large spread even without the tunnel. Queueing and contention are plausible contributors to the increase across all modes. The 91 ms gap between encrypted and plaintext loaded means cannot be attributed solely to AEAD from two repetitions. CPU averages also do not profile kernel work or AP queues. A controlled wired comparison, more repetitions and queue instrumentation would be needed to identify the dominant cause and estimate a reliable latency penalty.

7 Conclusion

The tunnel carries ordinary IPv4 traffic through a userspace TUN/UDP path and adds authenticated setup, confidentiality, integrity and bounded replay protection. The capture results place the encryption boundary at the expected interface. The most consistent performance finding is approximately five additional percentage points of process CPU in encrypted TCP mode. Encrypted sender throughput reaches 48.73 Mbps A→B and 45.01 Mbps B→A in the interleaved experiment.

Small UDP payloads expose a packet-rate constraint, while larger datagrams sustain the 20 Mbps offer. Idle RTT remains around 9–11 ms, but concurrent TCP traffic produces large and variable latency in all modes. These results support a practical educational tunnel on this testbed; they do not establish a universal cipher overhead or equivalence to a production VPN.

7.1 Protocol and operational limits

There is no key rotation, final handshake confirmation, setup cookie, peer roaming or multi-peer routing. Encryption does not hide traffic metadata or guarantee availability. The daemon relies on the configured TUN MTU and does not adapt to a smaller outer path.

7.2 Limits of the measurements

Five TCP trials, two UDP/load repetitions and fixed mode order provide a limited sample on shared Wi-Fi. Interleaving reduces time separation but cannot remove all contention or scheduling effects. Process CPU alone does not profile the complete network path.

8 Future Work

The next stage should strengthen session management and isolate performance costs before extending the tunnel to larger deployments.

8.1 Session lifecycle and handshake hardening

Add periodic rekeying with explicit session epochs, fresh directional keys and a bounded transition between old and new sessions. A final authenticated confirmation would let the responder verify that the initiator derived the session keys before treating setup as complete. A cookie mechanism and rate limits could reduce the work caused by repeated or spoofed setup requests. Tests should cover lost confirmations, rekey retries, stale packets and restarts without nonce reuse.

8.2 Controlled performance evaluation

Repeat the three-mode comparison on a wired link and then on Wi-Fi, using randomized or counterbalanced mode order and more trials. Record kernel drop counters, queue occupancy, retransmissions and process profiles alongside throughput and CPU. This would help locate the small-packet constraint and separate forwarding, cryptographic and wireless costs. Loaded-latency experiments should report per-run distributions and tail latency rather than relying only on two session averages.

8.3 Packet-path efficiency and MTU handling

Profile copying, socket calls, nonce construction and AEAD before optimizing the forwarding loop. Candidate changes include batching datagram I/O and reusing the session's direction-specific nonce prefix. Each change should be compared with the existing path under identical packet sizes and offers. Add explicit handling for a smaller outer path MTU and test both oversize rejection and recovery, while preserving one authenticated inner packet per datagram.

8.4 Operational robustness and broader support

Separate privileged setup from long-running forwarding more strictly, bound diagnostic output and improve recovery when a peer becomes unreachable. After the single-peer lifecycle is reliable, extend the design to IPv6 and multiple peers with explicit route ownership and independent keys, sequence counters and replay windows. These extensions require isolation tests so that one peer's state or traffic cannot affect another's authentication or routing.

References

- [1] The Linux Kernel documentation. *Universal TUN/TAP device driver*.
<https://www.kernel.org/doc/html/latest/networking/tuntap.html>.
- [2] J. Postel. *User Datagram Protocol*. RFC 768, August 1980. <https://www.rfc-editor.org/rfc/rfc768>.
- [3] Y. Nir and A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, June 2018.
<https://www.rfc-editor.org/rfc/rfc8439>.
- [4] S. Josefsson and I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, January 2017.
<https://www.rfc-editor.org/rfc/rfc8032>.
- [5] Libsodium documentation. *Key exchange*. https://doc.libsodium.org/key_exchange.
- [6] M.-J. O. Saarinen and J.-P. Aumasson. *The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)*. RFC 7693, November 2015. <https://www.rfc-editor.org/rfc/rfc7693>.
- [7] The LLVM Project. *AddressSanitizer*. Clang documentation.
<https://clang.llvm.org/docs/AddressSanitizer.html>.
- [8] The LLVM Project. *UndefinedBehaviorSanitizer*. Clang documentation.
<https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>.
- [9] Energy Sciences Network (ESnet). *Invoking iperf3*. <https://software.es.net/iperf/invoking.html>.